QR Code Accelerator in Bluespec

BSV design and functional verification report

CAD for VLSI 2026 project, RTL part. Version of 6 November 2026.

Team members and roll numbers: *(fill in before submission)*

Scope of this version. This report covers the microarchitecture of the decoder and its functional verification in Bluesim. Synthesis, design-space exploration and place-and-route results are not included yet; they are added only once they have been measured (section 8).

1. Summary

The decoder reads a QR bitmap through a memory port that returns one module (bit) per request, corrects errors with Reed-Solomon, and returns the message bytes with a `last` flag. It is built from seven units. Every unit has its own self-checking test bench and was compared against the supplied Python model; units 2, 4, 5a, 5 and 6 were also compared against an independent reference written from the spec. The whole decoder was then run against all 1,614 symbols of the model's vector set, one after another without reset, with random stalls on `getByte`.

Result. 1,614 of 1,614 symbols behave as expected (1,168 decode to the right bytes, 446 correctly raise `failed`), with zero mismatches and zero handshake errors, at buffer sizes of 300 and of 1,024 bytes. The slowest symbol takes 37,388 cycles at the 1,024-byte buffer, close to the single-pass reading time of the largest symbol (about 31,000 cycles), because the design is bound by the memory port.

Unit	What it does	Tests run (all pass)	Planted bugs caught
1 Walker + mask	Order of data modules, skips function modules; mask test	Positions equal the model for all 40 versions; all 8 masks equal over 177x177	not recorded (exhaustive comparison)
2 Format decoder	15+15 format bits to ok / level / mask	All 32,768 words (exhaustive); 1,200 positions; 400,000 random copy pairs	1 (tie rule): about 75k mismatches
3 Reader	Walker + format reads + memory port + unmask to codeword bytes	1,563 cases, about 1.98 M codeword bytes identical; 37 format failures; 14 bad sizes	2 of 2
4 Block buffer	De-interleave, group planner, block walker, read/write port	All 160 version/level pairs at buffer sizes 153, 300, 1,024	3 of 3
5a GF(2^8)	Multiply, inverse, constants	All 65,536 products, 256 inverses, constants	1 of 1 (63,901 mismatches)
5 Reed-Solomon	Syndromes, Berlekamp-Massey, Chien/Forney, verify	4,099 blocks in 416 cases (clean, 1..t errors, too damaged, near another codeword)	4 of 4 (see 5.4)
6 Payload parser	Bit stream to characters, last flag, failure rules	2,014 cases (895 decode, 1,119 fail), 20,021 characters	7 of 7
7 Top level	Controller, byte forwarding, C1-C5 interface	1,614 symbols, 0 mismatches at both buffer sizes	*(fill in result of mutants.sh)*

Table 1. Units and their verification results (numbers copied from the test outputs).

2. Architecture

The data flows from the memory port through the reader into a buffer of whole blocks, through Reed-Solomon, and into the payload parser. A single controller sequences the passes and blocks (Figure 1).

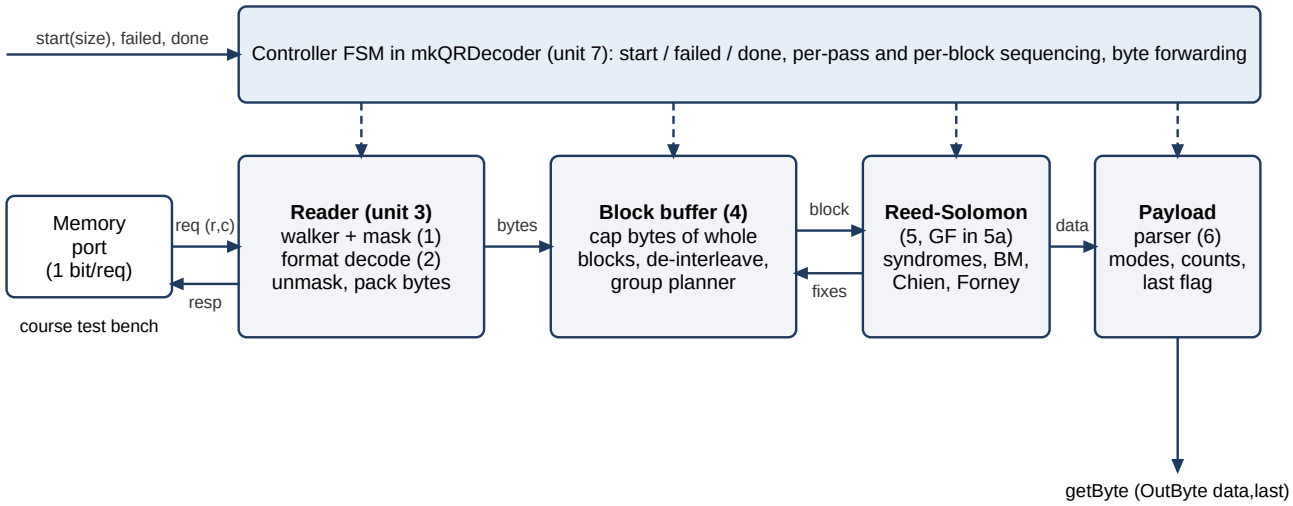

Figure 1. Block diagram of mkQRDecoder.

The central decision: where the codeword bytes are kept. A QR symbol stores its blocks interleaved, but the message has to come out with block 0 first, and the memory port has no cheap way to jump to the k-th module (re-reading means re-walking the whole symbol). Storing all bytes would need up to 3,706 bytes of flip-flops (29,648 bits for version 40). Instead the design stores only a *group* of whole blocks that fits in a buffer of `cap` bytes, and reads the symbol again for each further group. A larger buffer costs area and saves passes. This is the main area/time knob of the design; it is one type parameter in `QRBlockBuf#(cap)`.

cap (bytes)	flop bits	max passes per symbol	worst-case read cycles
153 (minimum)	1,224	34	1,060,528
300	2,400	22	625,504
512	4,096	9	280,728
1,024	8,192	5	155,960
2,048	16,384	2	62,384
3,706 (whole symbol)	29,648	1	31,192

Table 2. Buffer size against passes, worst case over all 160 version/level pairs (from the block-buffer test; one read pass costs about N(N-1)+40 cycles).

The memory port bounds the speed. One module is read per cycle and the walker also spends one cycle on each skipped function module, so one pass over a version 40 symbol takes 31,153 cycles (29,648 data modules). Reed-Solomon work is small next to that.

3. Microarchitecture of the units

3.1 Walker and mask (unit 1)

A pure function decides whether a module belongs to a function pattern (finders, timing, alignment, format, version areas) for a given version. The walker steps through the symbol in the specified two-column zigzag and offers only data modules; skipped modules cost one cycle each (version 1: 421 cycles for 208 modules; version 40: 31,153 for 29,648). A second pure function says whether the mask inverts a module, for the 8 mask patterns.

3.2 Format decoder (unit 2)

The 15-bit format word is a BCH code with 32 valid words and minimum distance 7, so up to 3 wrong bits can be corrected. `nearestFormat` compares a received word with all 32 valid words in parallel and returns the closest; `decodeFormat` applies it to copy 1 and copy 2, prefers copy 1 on a tie, and accepts the result only if its distance is at most 3. These are pure combinational functions. The 32 valid words in the supplied table were re-derived from the spec's BCH description as part of the test.

3.3 Reader (unit 3)

The reader first reads the 30 format modules, then walks the data modules, unmasks each one and packs eight modules into a byte (first bit is the MSB); the few leftover bits of some versions are dropped. The memory

answers one cycle after the request, so the reader saves the requested (row, column) and unmasks with that saved position. It relies on rule C4 (a request that is ready is taken in the same cycle) and has a one-position buffer between the walker and the port. A slow consumer stalls it without losing or repeating a bit. A full data read costs N(N-1)+1 cycles plus about 34.

3.4 Block buffer (unit 4)

Bytes arrive in symbol order and are written into the right block position using three counters (phase, block, index) and the remembered offset of the first long block, so no division is needed. A greedy planner picks the group of whole blocks that fits (at most 81 cycles per pass). A block walker hands the controller each block's base address, data length and total length. The storage is plain registers without reset, with a decoded row/column address and an AND-OR read tree, so the generated Verilog uses no library memory. A write port lets Reed-Solomon store its corrections.

3.5 GF(2^8) and Reed-Solomon (units 5a, 5)

The field uses the polynomial 0x11D with alpha = 2. `gfMul` is 8 AND-XOR rows and shrinks to a few XORs when one argument is a constant; `gfInv` computes a^{254} in parallel (13 multipliers' worth of logic).

Per block Reed-Solomon runs: (1) one read pass computing all syndromes (30 constant multipliers); if all are zero the block is clean and goes straight to output; (2) otherwise Berlekamp-Massey, failing if the error count L exceeds e/2; (3) the error-evaluator polynomial; (4) a Chien search with Forney's formula, which must find exactly L roots and writes each correction back into the buffer; (5) a verify pass in which all syndromes must be zero; (6) output of the data bytes. The baseline shares one multiplier and one inverter. Working storage is 95 bytes (760 flip-flops). Measured on the 4,099 test blocks, the slowest blocks took 527 cycles when clean, 2,003 when fixed, 2,059 when fixed with the maximum number of errors, and 1,459 when they failed.

3.6 Payload parser (unit 6)

The corrected data bytes form a bit stream, read MSB first as segments: a 4-bit mode (numeric 0001, alphanumeric 0010, byte 0100, terminator 0000), a count whose width depends on the version (versions 1-9, 10-26, 27-40), then the characters. The parser keeps a 24-bit right-aligned bit accumulator (a byte is accepted while fewer than 17 bits are buffered) and a state machine that takes one field at a time. A numeric group becomes up to 3 digits (10, 7 or 4 bits); an alphanumeric pair becomes 45a+b (11 bits) or a single character (6 bits); both go through a three-character staging vector and leave one per cycle. Because `last` may only be set on the final character, one character is always held back and released when the terminator or the end of data is seen. It fails on an unknown mode, a field that runs past the end, a numeric group above 999 (99, 9 for the short groups), or an alphanumeric pair above 2024 (single above 44). After the end of the message it keeps accepting and discarding bytes, so the producer never stalls.

3.7 Top level (unit 7)

One rule owns the controller state (Figure 2). `start` has the implicit condition that the decoder is idle, and `getByte` is the parser's output with the extra condition that `failed` is low (rules C3, C5). Two small rules forward bytes: reader to buffer while the buffer fills, and Reed-Solomon to parser while a block is corrected. `done` rises only after `endOfData` and when the parser has handed out every character.

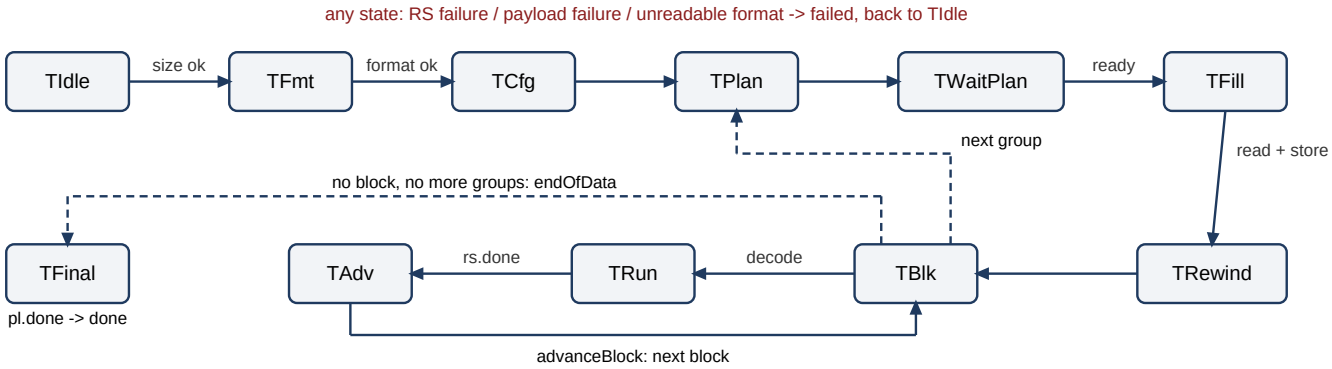

Figure 2. Controller states of mkQRDecoder.

Failure handling. A bad side length raises `failed` at once. An unreadable format, an uncorrectable block or a malformed payload raises it and returns to idle immediately. This is safe because reading and Reed-Solomon

never overlap (the reader is idle whenever a failure can happen) and every sub-unit restarts from any state, so the next `start` reconfigures everything. The parser emits the `last` character as soon as it has seen the terminator, which can be before later blocks are corrected; if one of them then fails, `failed` rises after bytes were returned, which rule C2 allows (see the open questions in section 7).

Structure. The reader is wrapped as a separate synthesized module (`mkQRReaderS`). Its format decoder is a large unrolled function, and inside the controller rule it used up bsc's elaboration step budget (warning G0024). As its own module it elaborates separately, and the generated Verilog then has two modules, `mkQRDecoder` and `mkQRReaderS`.

4. Verification method

Independent references. Where possible a checker compares the design with the Python model *and* with a reference computed from the spec in the checker itself (a payload parser, a Reed-Solomon encoder, the GF definition, the interleaving formula, the BCH words). When the two references disagree the run stops before the RTL is judged; this caught two mistakes in the payload checker's own case list.

Per-unit test benches are self-checking, run in Bluesim with one command, and print one summary line. Wherever the input space allows, the test is exhaustive (32,768 format words, 65,536 GF products); otherwise it combines random cases with hand-built hard ones.

Planted bugs. For each unit a few bugs are inserted into a scratch copy of the RTL and the test must fail. A bug that survives is reported and used to improve the test (an example is in 5.3).

Whole-decoder test bench (`TbTop.bsv`). It plays the part of the course test bench: it holds the bitmap, answers every `mem.req` one cycle later (reads outside the symbol return 0), calls `start` as soon as the previous symbol has ended, and decodes the symbols one after another without reset. Bytes are taken with four patterns (every cycle; about 1 cycle in 4; 1 in 32; 1 in 64), chosen by the number of the symbol so that all patterns occur in every category. While it runs it checks: C1, every byte and its `last` flag against the expected message; C2, that `failed` and `done` are never high together, that the first one to rise stays high until the next start, that `done` does not rise before the byte with `last` was taken, that a valid symbol never fails and a symbol that must fail never finishes; C3, that no byte is taken while `failed` is high. A symbol that produces no result within 3,000,000 cycles is an error. C5 is checked statically: the run first elaborates `mkQRDecoder` to Verilog and requires the ready signals of `start` and `getByte` to exist. C4 is satisfied by construction in the bench's memory (every ready request is taken in the same cycle).

5. Test set and results

5.1 What the test set contains

The vector set (1,614 symbols, 18,865 expected message bytes) is generated by the model. Counts below are measured from the generated case list.

Category (spec 5.3)	Symbols	Notes
Every version and level, damage at the limit	320	all 40 versions x 4 levels: t = floor(e/2) damaged bytes in every block (160); random damage (160)
Too many errors (must fail)	160	one damaged byte too many in a block; all 40 versions x 4 levels
Every mask and mode, clean symbols	160	alphanumeric 63, byte 49, numeric 48; all 40 versions, 4 levels and 8 masks occur
Short final groups	108	numeric with 1-5 digits (60), alphanumeric with 1-3 characters (36), short byte (12)
Several segments, empty, full capacity	48	multi-segment 12; empty message 12; empty numeric segment 12; full-capacity byte message 12
Format information with up to 3 wrong bits	360	wrong bits in copy 1 + copy 2: 0+3, 1+2, 2+3, 3+0, 3+1, 3+3 (60 symbols each)
One format copy destroyed	60	the other copy must be used
Format information past repair (must fail)	128	unreadable format
Damaged function modules	112	function-pattern damage 56; finder patterns erased 56
Bad side length (must fail)	14	sizes that are not 17 + 4v, or that do not match the given symbol

Category (spec 5.3)	Symbols	Notes
Malformed payload (must fail)	144	12 kinds x 12 symbols: unknown, ECI, kanji and structured-append mode; numeric group above 999, pair above 99, single above 9; alphanumeric pair above 2024, single above 44; byte, numeric and alphanumeric segment running past the end

Table 3. Test set of the whole-decoder run (rows add up to 1,614 symbols; the four must-fail rows give the 446 symbols that must raise failed).

5.2 Whole-decoder results

Run	Symbols	Mismatches	Latency, no back-pressure (symbols decoded to a non-empty message)
buffer 300, every 8th symbol	202 (146 decode, 56 fail)	0	36 symbols, max 31,375 cycles, mean 10,082
buffer 300, all symbols	1,614 (1,168 decode, 446 fail)	0	286 symbols, max 192,458 cycles, mean 11,826
buffer 1,024, all symbols	1,614 (1,168 decode, 446 fail)	0	286 symbols, max 37,388 cycles, mean 11,278

Table 4. Results of run.sh (Bluesim, bsc 2021.12.1). Latency is measured from the cycle `start` fires to the cycle the byte with `last` is taken.

The slowest symbol in both full runs is a version 27 level L symbol whose message fills all data bytes, so the last byte only comes out after the last block. It needs 12 passes of 15,500 module reads at the 300-byte buffer and 2 passes at 1,024, which accounts for the drop from 192,458 to 37,388 cycles. The next slowest symbols (about 32,900 cycles) are version 40 symbols with damage in every block; a single pass reading them already takes 31,153 cycles, so little more than 1,700 cycles are left for everything else. Note that the latency is measured to the byte with `last`. When a message ends in the first blocks, that byte is delivered after about one read pass, while all later blocks are still to be read and corrected before `done` rises, so the time to `done` is longer.

5.3 Bugs and problems found

Each item below was found by the tests or by bsc's own warnings, and fixed.

- **Payload test bench missed a stuck producer.** A planted bug that removed the 'keep accepting bytes after the end' rule survived, because the bench ended a symbol as soon as `done` rose without checking that all bytes had been accepted, and the vectors rarely had enough bytes after the terminator. Fix: the bench now requires all bytes to be accepted, and 80 short messages with tails were added. The same bug is now caught by 23 symbols in the logic model and by the planted-bug run.
- **Rule conflicts in test benches.** Two rules that each read what the other writes made bsc choose one rule that could then never fire (warnings G0010, G0021); the simulation never ended. Fix: free-running counter plus a start timestamp, and ConfigReg for registers read across rules. The same kind of conflict was found earlier in the Reed-Solomon bench.
- **Elaboration step budget** (G0024) when the reader was inlined into the controller; fixed by the separate reader module (section 3.7).
- **Test case mistakes.** The payload checker's generator reported two cases in its own case list that did not behave as intended (a wrong version, and padding bits that completed a field), before any RTL was judged.
- **Design bugs.** The tests found no functional bug in the payload parser or the top level: their first complete simulations passed once the test-bench problems above were fixed. The planted-bug runs show which failure classes the checkers would have caught.

5.4 Planted bugs, units 5 and 6

Reed-Solomon (4 caught): Chien-search direction reversed; Berlekamp-Massey length test off by one; correction written one byte off; both safety checks removed. Removing only the check for 'exactly L roots' is *not* caught because the verify pass catches the same blocks; the roots check only rejects earlier, so both checks are kept. Payload parser (7 caught): count-width boundary 26 to 27; numeric limit 999 to 1,023; alphanumeric limit 2,024 to 2,025; `last` never set; fewer than 4 leftover bits treated as a failure; bytes after the end not discarded; byte-mode count width 16 to 8.

5.5 Not covered yet

- Code coverage (Verilator `--coverage` or cocotb) was not measured; coverage so far is the category counts in Table 3.
- The planted-bug run for the top level is *(to be filled in)*.
- Verification is in Bluesim only; the generated Verilog has been elaborated but not simulated.

6. Assumptions

The spec leaves some choices open. The model and the RTL make these choices, and the reference checkers agree with them:

- a segment with count 0 is legal;
- fewer than 4 bits left with no terminator ends the message successfully;
- numeric groups keep their leading zeros;
- an unknown mode means anything other than 0001, 0010, 0100 or 0000;
- a failure in any block fails the whole symbol, even if all message bytes were already delivered.

Bytes delivered before `failed` rises are discarded by the test bench, as rule C2 describes.

7. Open questions

- What is the cycle limit per decode? It decides the buffer size, because a smaller buffer saves area but needs more passes.
- Are the choices in section 6 correct?
- Is `failed` rising after the byte with `last` was taken acceptable? The decoder does this when a later block cannot be corrected.

8. Remaining work

Synthesis with Design Compiler (normal synthesis, 40% external delay), at least four iterations with Table 5 of the spec for each, and place-and-route of the best design are still to be done; no synthesis numbers are claimed in this report. The levers identified from the design so far are: the buffer size (area against cycles, Table 2); the format decoder (32 comparisons at once against one per cycle); the Reed-Solomon multiplier and inverter (one shared multiplier against several, serial inverse, fewer syndrome multipliers); the walker (look ahead over function modules); and the clock period.